RACS Project

Nefeli Kasa - 7115112500022

18/06/2026

1 Introduction

1.1 Project Scope

This project is based on Redundant Array of Cloud Storage (RACS), as described in the 2010 paper by Hussam Abu-Libdeh et al. here. A base version of the RACS system is implemented in this work, utilizing three different cloud providers (AWS, Microsoft Azure, Google Cloud) for data stripping and offering an interface that supports the *put, update, get, delete* operations. Aspects that were omitted in this initial effort include cross-cloud synchronization using Zookeeper, asynchronous operations and policy hints.

1.2 Problem Description

A problem that often arises in modern cloud storage services is vendor lock-in. Specifically, the paper coins the term storage inertia to describe the difficulty clients face in switching providers. This issue stems from conscious provider policies to overcharge for inbound and outbound data requests, while keeping storage costs relatively low. Mitigating this requires data dispersion across multiple providers. Adding this data redundancy can be beneficial for tolerating temporary outages or data loss, while also enabling flexible pricing schemes where new or cheaper cloud providers are preferred for data access and expensive providers are avoided altogether.

1.3 Proposed Solution

RACS aims to solve the problem of vendor lock-in in the cloud storage domain by offering clients a proxy service that disperses their data among multiple cloud providers. To achieve this, mechanisms inspired by RAID techniques are employed to stripe data across various providers while simultaneously introducing redundancy for fault tolerance. RACS acts as an intermediary between the client and n diverse cloud repositories, transparently modifying all operations to invoke multiple repositories at once. Redundancy is ensured through erasure coding (e.g., Reed-Solomon), where m original fragments are mapped to a larger

set of n fragments, adhering to the property that the original data can be fully reconstructed from any m of the n fragments.

2 RACS Architecture

RACS is built on a microservices architecture. Each component is hosted in independent Kubernetes pods and communicates with the rest of the cluster via exposed APIs. This architecture was chosen as it is an industry standard for distributed systems; it allows each service to operate independently, enhancing the modularity, fault tolerance, and allowing for independent, horizontal scalability of high-demand components.

The system was deployed in Kubernetes, an orchestration tool for managing containerized workloads and services. Kubernetes was chosen due to its vast array of essential services when managing loosely couple services, such as scheduling, automatic scaling, service discovery and load balancing, self healing, declarative configurations and extensibility via custom resources and others.

2.1 Kubernetes Custom Resource

To provide a declarative configuration for all core RACS properties, a Kubernetes Custom Resource (CR) was established:

```
schema:
  openAPIV3Schema:
    type: object
    properties:
      spec:
        type: object
        properties:
          selection_policy:
            type: string
            enum: ["random", "load-balancing", "latency"]
            default: random
          encoding_parameter_k:
            type: integer
            default: 2
          encoding_parameter_n:
            type: integer
            default: 3
```

As illustrated in the schema above, the configurable parameters include:

- **Encoding parameters (n, k)**: Here, n represents the total number of distributed shards, and k denotes the minimum number of shards required to fully reconstruct the original data.

- **Selection policy**: This dictates the strategy used to select k out of n providers to fulfill each request. While currently limited to the default `random` policy, future implementations (such as latency or load-balancing) could yield significant performance and cost benefits by leveraging gathered telemetry data.

A dedicated RACS controller actively monitors this custom resource. Upon the application of a new configuration manifest, the controller automatically provisions all microservices and underlying Kubernetes resources required to support the system's operation.

2.2 Core Components

The main system components of this RACS implementation are:

- **RACS Server:** The top layer and single point of client interaction. It exposes a Python-based API (FastAPI) to handle all fundamental operations a client needs to interact with the storage service (login, put, update, get, and logout). Because asynchronous operations were omitted in this iteration, the server executes these operations synchronously, leveraging FastAPI's internal thread pool to manage concurrent requests without blocking the main server execution.

- **Codex Service:** Responsible for encode and decode operations. It utilizes the zfec Python library to apply erasure coding, enhancing data chunks with parity information. This is the core engine behind the RACS premise, enabling full data recovery from only a subset of the encoded chunks.

- **Redis Key-Value Store:** Manages all stateful data required by the system, such as user credentials. By offloading state management to Redis, the core RACS server remains strictly stateless, allowing it to seamlessly scale horizontally.

- **AWS/Azure/GCP Service**: Each external cloud provider is abstracted into a dedicated microservice. These microservices receive requests from the RACS server and fulfill them using the native Python SDKs of their respective platforms.

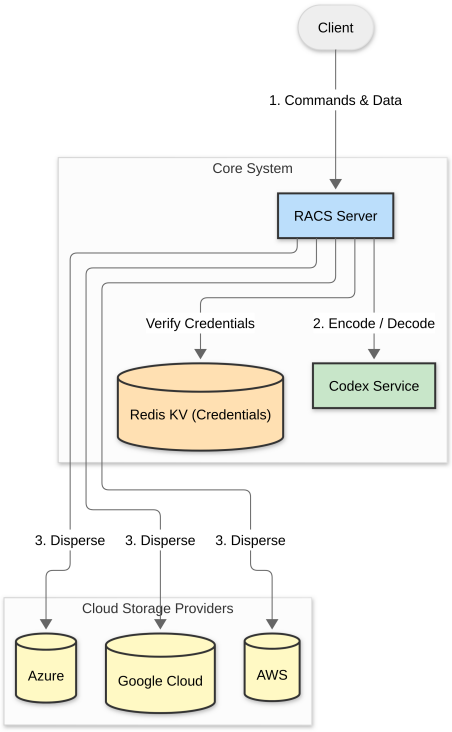

Figure 1: RACS Block Diagram

2.3 Security

To manage security and access control, the system implements JSON Web Tokens (JWT). Upon successful verification of user credentials during the initial login, the server issues a cryptographically signed JWT to the client. This token is subsequently embedded into the authorization header of all outgoing HTTP requests. This mechanism allows the server to authenticate and authorize the client for protected operations without repeatedly transmitting sensitive credentials.

3 Demonstration

3.1 Execution Environment

The RACS system was containerized using Docker and deployed on a three-node virtualized Kubernetes cluster, provisioned via Minikube.

3.2 Results

To validate the system's core capabilities, an automated script executed the following sequence of operations using a 1.3 MB text file as the test payload:

1. Login
2. Put
3. Get
4. Update
5. Get
6. Logout

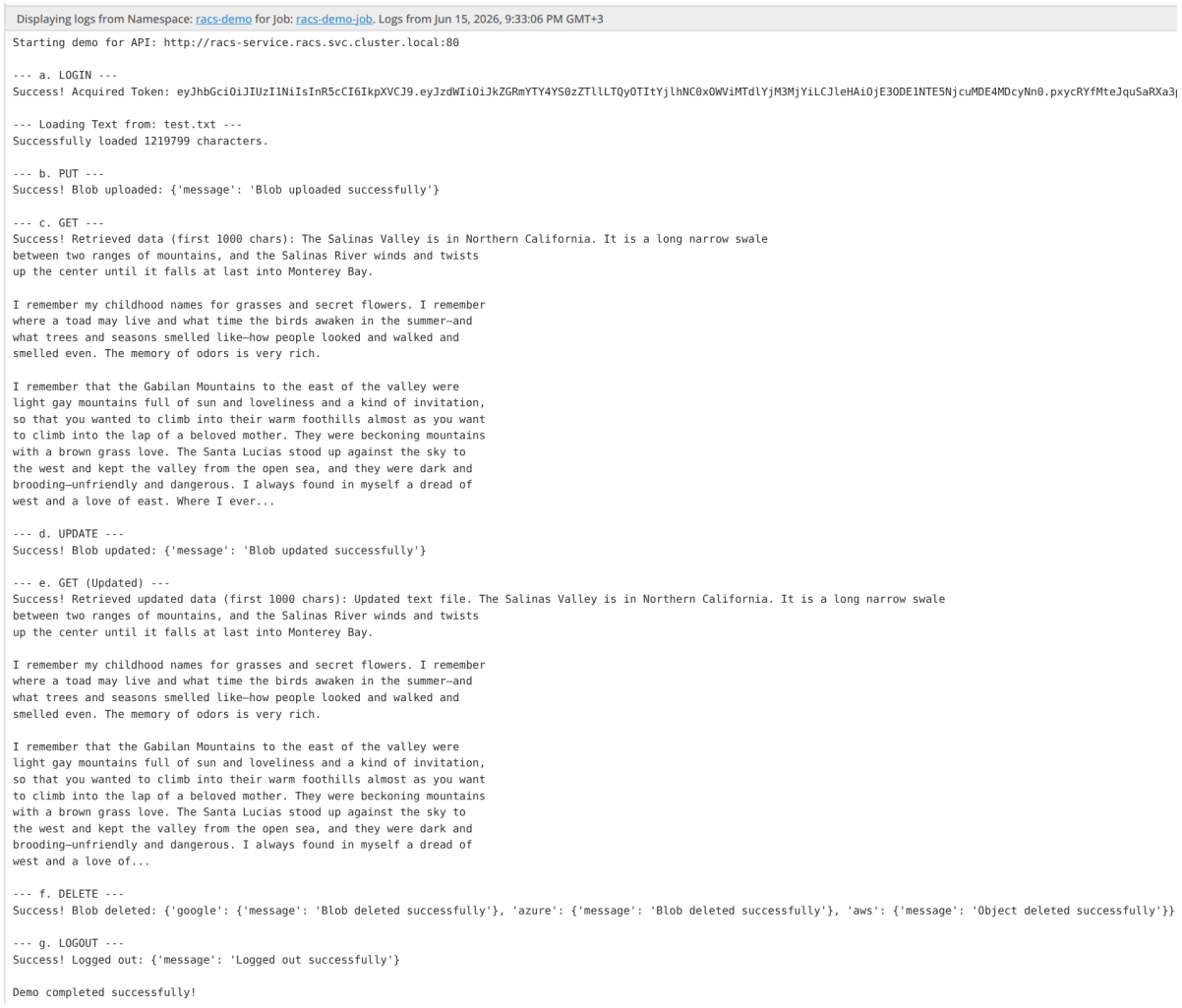

Figure 2: Test Results

As illustrated in the captured snapshots, all operations executed successfully, with the entire script completing in under thirty seconds.

Furthermore, the RACS server logs confirm that the microservices successfully managed all interactions with the respective cloud providers. Crucially, the logs demonstrate that while put operations trigger calls to all three providers, get operations require responses from only two. This validates the core premise of the architecture: full data reconstruction is successfully achieved using only a subset (two) of the distributed parity shards.

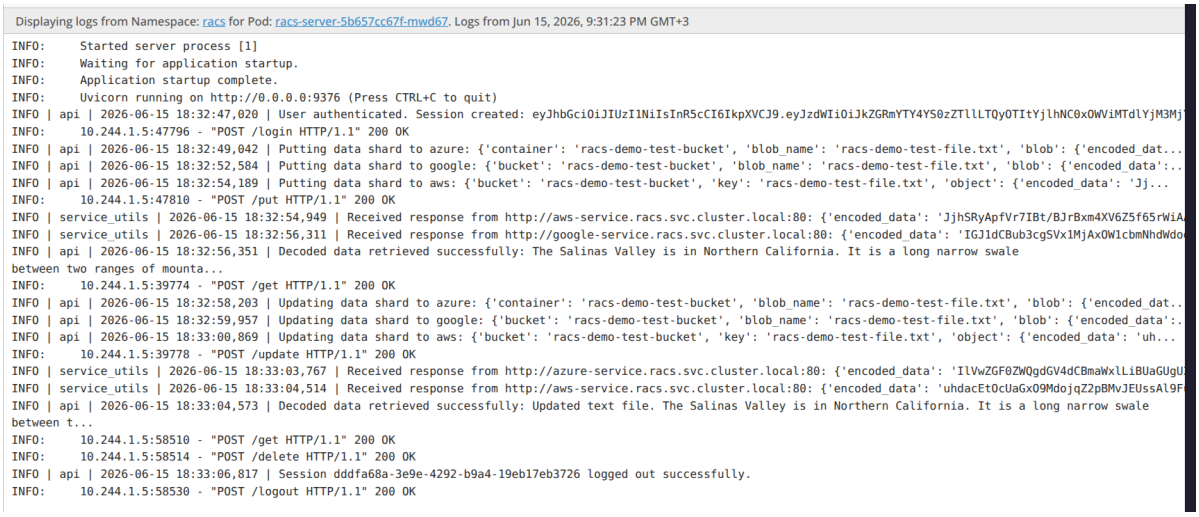

Figure 3: RACS Server Logs

4 Code Availability

The complete source code for the implemented RACS system is available on Github at the following link.